Hybrid Quantum-Classical Neural Networks

EVA: Quantum Machine Learning – Lecture 9

Pavel Sulimov

Spring 2026

ZHAW Zürcher Hochschule für Angewandte Wissenschaften

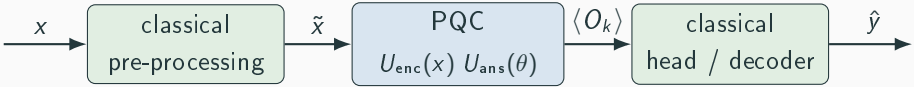
Roadmap

1. Hybrid architecture: anatomy and design space
2. PQC zoo: which ansatz for which task
3. Encoding in practice: matching data type to encoding strategy
4. Classical–quantum interplay: C-Q, Q-C, C-Q-C, Q-C-Q-C
5. PyTorch integration: `TorchLayer` and `nn.Module`
6. Training advice and failure modes

Part 1. Hybrid architecture: anatomy

The hybrid paradigm

A hybrid quantum neural network (QNN) combines a parameterized quantum circuit (PQC) with classical pre- and post-processing blocks:

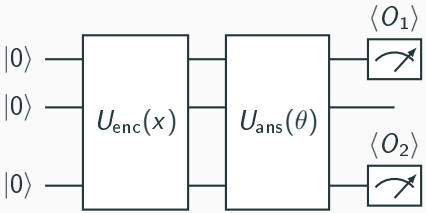


The pre-processing block is optional (may be identity). The post-processing block is optional (expectation value may be used directly).

The PQC has two distinct parameter families:

- data parameters x: fixed at inference, enter through the encoding layer
- trainable parameters θ: optimized during training, enter through the ansatz

Inside the PQC: circuit anatomy



Encoding layer $U_{\text{enc}}(x)$: maps classical features to a quantum state; no free parameters.

Ansatz $U_{\text{ans}}(\theta)$: parameterized unitary; rotations and entanglers with trainable angles θ.

Measurement: expectation values $\langle O_k \rangle = \langle \psi | \, O_k \, | \psi \rangle$ or output probabilities $p(z)$.

Output types

Three common output types and their typical uses:

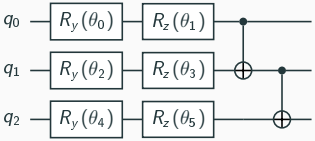
Output	Formula	Typical use
Single expectation	$\langle Z_0 \rangle \in [-1, 1]$	binary classification, regression
Multiple expectations	$(\langle O_1 \rangle, \ldots, \langle O_m \rangle)$	multi-class, feature extraction
Output probabilities	$(p_0, \ldots, p_{2^n-1})$	generative models, sampling tasks

In practice, measuring $\langle Z_0 \rangle$ and passing it through a sigmoid is the most common recipe for binary classification. Measuring $\langle Z_0 \rangle$ on each of m qubits gives m independent features for a classical head.

Part 2. PQC zoo: ansatz families

Hardware-efficient ansatz (HEA)

The most common ansatz for noisy intermediate-scale quantum (NISQ) devices. One layer alternates single-qubit rotations with a fixed entangling pattern:



Strengths:

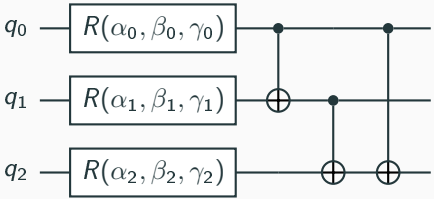
- shallow and hardware-native (CNOT topology matches typical devices)
- easy to scale: $2 \times n \times L$ parameters for n qubits, L layers

Caveats:

- no task-specific structure; expressibility is generic
- prone to barren plateaus at large depth

Strongly entangling layers (SEL)

Used in PennyLane's `StronglyEntanglingLayers`. Every pair of qubits gets entangled at each layer with varying range:

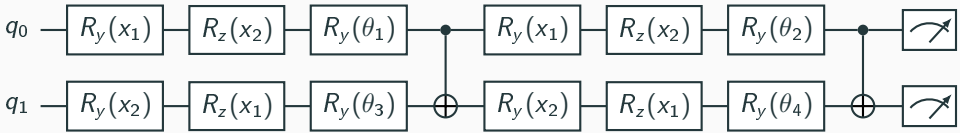


Each qubit gets a full $SU(2)$ rotation (3 parameters) plus long-range CNOT connections.

Good default when you want maximum expressibility at modest qubit count. More expensive than HEA in circuit depth.

Data re-uploading circuit

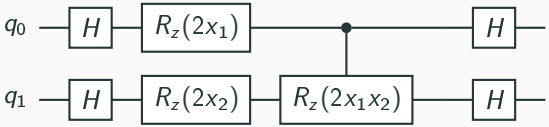
Data is injected at multiple points, interleaved with trainable layers. This is the architecture of choice when the function to approximate requires higher Fourier frequencies:



Repeating data injection allows the circuit to represent functions with more harmonics than a single-pass encoding would permit. The number of accessible frequencies grows linearly with the number of re-uploading layers.

IQP / ZZ-feature-map style

Instantaneous Quantum Polynomial (IQP) circuits interleave Hadamard layers with diagonal phase operators. Qiskit's `ZZFeatureMap` follows this pattern:



The phase $2x_1x_2$ encodes a pairwise interaction directly into the state.

Typically used as the encoding layer, not as a trainable ansatz. Combines well with a separate HEA ansatz block appended after it.

Rules of thumb: ansatz selection

Situation	Recommended ansatz	Why
Exploratory / first trial	HEA (2–4 layers)	Shallow, fast, no assumptions
Need strong correlations	Strongly entangling layers	Full $SU(2)$ + long-range CNOT
Non-linear function learning	Re-uploading (2–4 reps)	More Fourier harmonics per qubit
Problem-inspired structure (e.g. chemistry)	UCCSD, kUpCCGSD	Preserves physical symmetries
Binary classification	1-qubit re-uploading	Single qubit can be universal for 1D data
Transfer from classical CNN (conv. net)	Patch / convolutional QNN	Matches local receptive fields

Rules of thumb: practical guideline

Start with HEA.

Add re-uploading if training saturates before the loss converges. Re-uploading extends the frequency range of learnable functions and is low-cost to add: it reuses the same data injection code, just repeated between ansatz blocks.

Switch to a problem-inspired ansatz — such as UCCSD (Unitary Coupled Cluster Singles and Doubles) for molecular energy, or QAOA (Quantum Approximate Optimization Algorithm) for combinatorial tasks — only when there is clear evidence that the problem has physical structure that HEA cannot encode compactly.

When in doubt: measure the gradient norms at initialization. If they are near zero across all parameters, the ansatz is too deep or too wide for your qubit count – reduce depth before changing the architecture.

Part 3. Encoding in practice

Data type drives encoding choice

The encoding circuit is the interface between classical data and the quantum Hilbert space. Wrong encoding makes learning impossible regardless of ansatz quality.

Data type	Encoding	Practical note
Low-dim continuous (tabular)	Angle	One qubit per feature; normalize to $[0, \pi]$
High-dim, pre-normalized	Amplitude	$d = 2^n$; deep state prep circuit
Binary / discrete	Basis	Direct $\{0,1\}^n \to \lvert z\rangle$
Sequential / time-series	Re-uploading	Features re-injected at each layer
Image patches	Convolutional / patch	Classical CNN features $\to$ angle encoding

Angle encoding in practice

Canonical recipe for d-dimensional tabular features:

$$U_{\text{enc}}(x) = \bigotimes_{i=1}^{n} R_y(\pi\,\tilde{x}_i)\,, \qquad \tilde{x}_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}.$$

Scaling to $[0, \pi]$ avoids aliasing: nearby feature values map to nearby angles on the Bloch sphere, which means the induced kernel has meaningful resolution.

Common extensions:

- use both R_y and R_z to encode two features per qubit at the cost of less geometric separation
- ZZ-style interaction: apply $R_z(x_i x_j)$ after CX to inject pairwise features into the state without extra qubits

Rule: When in doubt, angle-encode into $[0, \pi]$ and measure the circuit expressibility before trying amplitude encoding.

Amplitude encoding: when it makes sense

Amplitude encoding stores 2^n values in n qubits, giving an exponential compression ratio in principle:

$$|\psi(x)\rangle = \sum_{j=0}^{2^n-1} \tilde{x}_j \, |j\rangle \,, \qquad \|\tilde{x}\| = 1.$$

In practice, this encoding is useful when:

- classical dimensionality reduction (PCA) already produces a normalized dense vector that happens to fit 2^n exactly
- the downstream operation genuinely exploits the full Hilbert-space structure (e.g. quantum distance estimation)

It is **not** a free speedup. Generic state preparation circuits are deep and may negate any quantum advantage on NISQ hardware.

On a simulator: `qml.AmplitudeEmbedding` or `StatePrep` in Qiskit. On hardware: generic state prep blows up circuit depth for $n > 4$.

Re-uploading for sequential and structured data

For a time series $(x^{(1)}, \ldots, x^{(T)})$, re-uploading treats each time step as a new encoding injection:

$$\underbrace{U_{\text{enc}}(x^{(1)})}_{\text{step 1}} \; U_{\text{ans}}(\theta^{(1)}) \; \underbrace{U_{\text{enc}}(x^{(2)})}_{\text{step 2}} \; U_{\text{ans}}(\theta^{(2)}) \; \cdots$$

Analogy to classical RNNs: each ansatz block acts as a transition function mixing the current quantum state with the next input.

Key property: the expressible function class grows with the number of re-uploading steps. Specifically, for a single-qubit circuit with L uploads of a scalar x, the accessible output functions span trigonometric polynomials of degree $\leq L$.

Demonstrated in `lecture_09_demo.ipynb`, Demo 3.

PennyLane datasets: practice reference

PennyLane provides pre-built datasets for benchmarking quantum machine learning (QML) models at `https://pennylane.ai/datasets/`.

Useful datasets for this lecture:

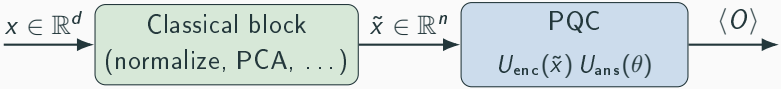
Dataset	Type	Suggested encoding
`qchem/H2`	molecular Hamiltonians	problem-inspired (UCCSD)
`other/bianconi-barabasi`	graph features (4–6D)	angle on normalized features
`other/digits`	8×8 greyscale images	amplitude (64 features, 6 qubits) or patch
Synthetic moons / blobs	2D, low-dim	angle, 2 qubits sufficient

For the practice session we generate synthetic 2D datasets (moons, circles, blobs) so no download is required.

Part 4. Classical–quantum interplay

Pattern A: C-Q

Classical pre-processing → Quantum core



When to use:

- raw data dimensionality exceeds available qubit count
- classical normalization or embedding is well understood (e.g. word2vec features into a QNN)
- want to leverage classical deep networks for feature extraction

Pattern B: Q-C

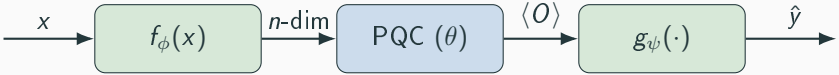
Quantum core → Classical head

When to use:

- the quantum circuit produces multiple observables that are collectively rich but individually noisy (averaging over heads improves stability)
- want to keep the quantum part minimal and let the classical head learn the output mapping
- **most common pattern in practice**; the classical head adds very few parameters but greatly improves final accuracy

Pattern C: C-Q-C (sandwich)

Classical encoder → Quantum → Classical decoder



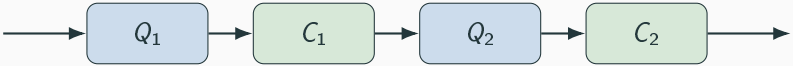
When to use:

- input is high-dimensional (classical encoder reduces to qubit count)
- output requires post-processing (e.g. multiple tasks, sequence output)
- the classical blocks carry the burden of dimensionality mismatch while the quantum block provides the non-linear transformation kernel

Example: ResNet feature extractor → 4-qubit QNN → softmax head for image classification.

Pattern D: Q-C-Q-C (alternating)

Multiple quantum and classical blocks, deeply interleaved



When to use:

- want the classical blocks to act as adaptive readout networks between quantum layers (mid-circuit measurement + re-encoding)
- mimics deep classical-quantum neural networks proposed in recent literature

Practical warning: the more quantum-classical interfaces you introduce, the more gradient flows are broken by measurement collapse. Each quantum-classical boundary is a non-differentiable sampling step unless you use the parameter-shift rule or backprop-mode simulation.

When does adding a classical part help?

Classical layers are **free in simulation** and almost free in inference. They should be added when:

Situation	Recommendation
$d \gg n$ (more features than qubits)	Classical encoder before encoding layer
PQC produces m noisy observables	Classical linear head after quantum block
Need calibrated output probabilities	Batch normalisation (BatchNorm) or temperature scaling after quantum
Training is unstable (vanishing gradient)	Add classical skip connection or residual branch
Dataset is small (<500 samples)	Prefer classical-only baseline; quantum adds variance

A linear classical head after the quantum block nearly always helps without adding meaningful classical capacity.

Part 5. PyTorch integration

PennyLane + PyTorch: the two paths

PennyLane supports two routes for PyTorch integration:

Route 1: `qml.qnn.TorchLayer`

Wraps a QNode into a `torch.nn.Module`-compatible layer. Works inside `nn.Sequential` and standard PyTorch training loops. Trainable circuit parameters are registered as `nn.Parameter` tensors.

Route 2: direct QNode in `nn.Module.forward`

Call the QNode directly in `forward()`, using `interface='torch'`. Provides more flexibility but requires manual parameter registration.

For most use cases, `TorchLayer` is the right default. Use the direct path only when you need non-standard parameter sharing or dynamic circuit construction.

Defining a QNode for TorchLayer

Step 1: define the QNode

```
dev = qml.device("default.qubit", wires=n_qubits)

@qml.qnode(dev, interface="torch")
def circuit(inputs, weights):
    qml.AngleEmbedding(inputs, wires=range(n_qubits))
    qml.StronglyEntanglingLayers(weights,
                                 wires=range(n_qubits))
    return [qml.expval(qml.PauliZ(i))
            for i in range(n_qubits)]
```

The function signature must follow the convention: first argument is data (`inputs`), subsequent arguments are trainable parameters (`weights`).

Wrapping with TorchLayer

Step 2: wrap and compose

```
weight_shapes = {"weights": (n_layers, n_qubits, 3)}

qlayer = qml.qnn.TorchLayer(circuit, weight_shapes)

model = nn.Sequential(
    nn.Linear(n_features, n_qubits),
    nn.Tanh(),
    qlayer,            # returns n_qubits floats
    nn.Linear(n_qubits, n_classes),
)
```

After wrapping, `qlayer` behaves exactly like any other `nn.Module`. Gradients flow through the circuit via the parameter-shift rule automatically. The `optimizer.step()` call handles both classical and quantum parameters.

Training loop: no special treatment needed

Step 3: standard PyTorch training

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn   = nn.CrossEntropyLoss()

for epoch in range(n_epochs):
    optimizer.zero_grad()
    logits = model(X_train)
    loss   = loss_fn(logits, y_train)
    loss.backward()        # computes all gradients
    optimizer.step()       # updates all parameters
```

`loss.backward()` triggers parameter-shift gradient computation for the quantum layer and standard backpropagation for classical layers. Both gradients are accumulated and applied by `optimizer.step()`.

Practical TorchLayer notes

Issue	Advice
Slow training	Use `diff_method='backprop'` in simulation for 10–100× speedup
Batch size	Pass 2D input tensors; TorchLayer handles batching internally via `vmap`
Multiple observables	Return a list of `qml.expval` for multi-output circuits
Freezing quantum params	Use `param.requires_grad = False` on specific parameters
Custom ansatz	Any valid QNode works; not limited to built-in templates

Hybrid model as a custom nn.Module

Sometimes `nn.Sequential` is not flexible enough. Custom `nn.Module` allows arbitrary forward logic:

Custom hybrid module

```
class HybridQNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.pre   = nn.Linear(d_in, n_qubits)
        self.qnode = qml.qnn.TorchLayer(circuit, shapes)
        self.post  = nn.Linear(n_qubits, n_classes)

    def forward(self, x):
        x = torch.tanh(self.pre(x))    # classical
        x = self.qnode(x)              # quantum
        return self.post(x)            # classical head
```

Part 6. Training advice and failure modes

Gradient methods: parameter-shift vs backprop

Two differentiation strategies are available in PennyLane:

Parameter-shift rule (default on hardware and shot-based simulation)

$$\frac{\partial\langle O\rangle}{\partial\theta_k} = \frac{1}{2}\left[\langle O\rangle\left(\theta_k + \tfrac{\pi}{2}\right) - \langle O\rangle\left(\theta_k - \tfrac{\pi}{2}\right)\right].$$

Each parameter requires two additional circuit evaluations. Exact gradient regardless of shot count (with noise floor from shot variance).

Backpropagation (simulation only, `diff_method='backprop'`) Computes exact gradient in a single forward + backward pass through the statevector. Only available with `default.qubit` or compatible simulators. Orders of magnitude faster for training on a simulator.

Rule: use `backprop` for all simulator training; switch to `parameter-shift` only when running on real hardware or a shot-based simulator.

Shot noise and batch size

When using a finite number of shots, each expectation value estimate has variance

$$\mathrm{Var}\left[\hat{O}\right] = \frac{1 - \langle O \rangle^2}{S},$$

where S is the number of shots.

Practical implications:

- with $S = 1024$ shots, the standard deviation of $\langle Z \rangle$ is at most $1/\sqrt{1024} \approx 0.03$; this is tolerable for classification
- gradient estimates via parameter-shift inherit this variance, so effective gradient noise is $\mathcal{O}(1/\sqrt{S})$
- increasing batch size is often more effective than increasing shots per sample when using shot-based simulators

For initial experiments on a simulator: use exact statevector (`shots=None`). Introduce finite shots only when studying shot-noise robustness explicitly.

Barren plateaus: the main risk

In randomly initialized circuits with n qubits and depth L, the gradient variance decays exponentially:

$$\mathrm{Var}\left[\frac{\partial \mathcal{L}}{\partial \theta_k}\right] = \mathcal{O}\left(2^{-n}\right).$$

Practical symptoms:

- loss is constant for many epochs regardless of learning rate
- gradients are all near zero at initialization
- increasing depth makes things worse, not better

Mitigations (covered in Lecture 11):

- use structured initialization (identity or small-angle init)
- keep circuit depth shallow relative to qubit count
- measure only a subset of qubits; keep other qubits unmeasured

Practical training checklist

Before declaring a model "not working", verify:

Check	Target value
Gradient norms at init	$> 10^{-4}$ for all parameters
Feature scaling	features in $[0, \pi]$ or $[-1, 1]$
Encoding aliasing	no two training samples map to same state
Ansatz depth	$\leq$ 3–4 layers for $n \leq 6$ qubits
Learning rate	start at 10^{-2}; reduce on plateau
Batch size	≥ 16; quantum layers benefit from batching
Baseline comparison	always run a logistic regression on the same split

A QNN that underperforms logistic regression on low-dimensional data is almost always a training problem, not an expressibility problem.

QNN versus classical: when to expect a difference

Expectation depends heavily on hardware era, qubit count, and task structure. "Classical outperforms" refers to a well-tuned logistic regression or neural network on the same train/test split.

Setting	Expectation
$n \leq 4$ qubits, standard tabular data	Classical outperforms or matches
Kernel-expressible tasks, data fits 2^n	Quantum kernel can match radial basis function SVM (RBF-SVM)
Fault-tolerant era, large circuits	Potential advantage for specific structures
NISQ, noisy hardware	Classical almost always wins on accuracy
Toy benchmarks in simulation	Results are optimistic; hardware not guaranteed

Why hybrid QNNs matter now

On current NISQ hardware, a QNN rarely outperforms a well-tuned classical model on standard benchmarks. The primary value of hybrid QNNs at this stage is:

- building intuition for the QML research pipeline: encoding, gradient estimation, and noise characterisation all have no classical analogues
- testing circuit architectures at small scale before error-corrected hardware arrives
- exploring whether quantum-native data (molecular energies, physical simulation outputs) benefits from quantum-native models

Summary

Topic	Key takeaway
Architecture	C-Q-C sandwich is the most practical hybrid pattern
Ansatz choice	Start HEA; add re-uploading if saturation; use problem-inspired rarely
Encoding	Angle (tabular); amplitude (dense pre-normalised); re-uploading (sequential)
Classical parts	Add a linear head; add classical encoder if $d \gg n$
Torch integration	TorchLayer wraps any QNode; training loop is standard PyTorch
Gradients	Backprop on simulator; parameter-shift on hardware
Failure modes	Check gradients first; barren plateaus are common at depth > 3

Lecture 10 will extend this to convolutional and equivariant quantum circuits, where the ansatz structure is chosen to match symmetries in the data.